

Understand your Module

1. Authentication & Account Module (Account)

This module manages user identity, password hashing, session creation via Cookie Authentication, role assignment, and access control redirects.

Code File Inventory

- **Controller:**

AccountController.cs

- **Domain Model:**

Login.cs

- **ViewModels:**

LoginViewModel.cs,

RegisterViewModel.cs

- **Views:**

Views/Account/Login.cshtml,

Register.cshtml,

AccessDenied.cshtml

- **Enum:**

UserRole.cs

- **Database Table:** Logins

Code Implementation & Data Flow

1. Login Processing (AccountController.Login):

- Accepts LoginViewModel (Username, Password).
- Queries _context.Logins via EF Core for a matching AssociatedName or email.
- Uses ASP.NET Core PasswordHasher<Login>.VerifyHashedPassword(...) to validate credentials against the stored PBKDF2 hash.
- Constructs a ClaimsIdentity containing:
 - ClaimTypes.NameIdentifier → user.UserId
 - ClaimTypes.Name → user.AssociatedName
 - ClaimTypes.Role → user.Role.ToString()
- Calls HttpContext.SignInAsync(...) to write the encrypted authentication cookie.
- Redirects to role-specific default routes configured in Program.cs (/Admin/Dashboard, /Shipper/Dashboard, etc.).

2. Registration Processing (AccountController.Register):

- Accepts RegisterViewModel (FullName, Email, Password, Role). Only allows public registration for Shipper or Consignee roles.
 - Hashes the password using PasswordHasher<Login>.HashPassword(...).
 - Saves new Login entity to _context.Logins.
-

2. Admin & Employee Administration Module (Admin)

This module enables system administrators to manage internal staff profiles, create user credentials, filter/page employees, and view high-level system metrics.

Code File Inventory

- **Controller:**

AdminController.cs

- **Service & Interface:**

IEmployeeService.cs,

EmployeeService.cs

- **Domain Models:**

Employee.cs,

Login.cs

- **ViewModels:**

AdminDashboardViewModel.cs,

EmployeeViewModel.cs,

EmployeeListViewModel.cs

- **Views:**

Views/Admin/Index.cshtml,

Employees.cshtml,

CreateEmployee.cshtml,

EditEmployee.cshtml

- **Database Tables:** Employees, Logins

Code Implementation & Data Flow

1. Paged Listing (EmployeeService.GetEmployeesPagedAsync):

- Applies LINQ filtering on _context.Employees.Include(e => e.Login) using search terms (name/email) and role filters.
- Dynamic sorting (sortBy, isDescending) and pagination using .Skip((page - 1) * pageSize).Take(pageSize).

2. Employee Creation (EmployeeService.CreateEmployeeAsync):

- Checks email uniqueness via `_context.Employees.AnyAsync(e => e.email == email)`.
- Creates a linked Login record with hashed initial password.
- Creates the Employee entity linking `userId` to the newly generated `Login.UserId` in an EF Core transaction.

3. Shipment Booking Module (ShipmentBooking)

Handles customer shipment orders, origin/destination validations, booking edits, cancellations, and compliance document uploads.

 Code File Inventory

- **Controller:**

ShipmentBookingController.cs

- **Service & Interface:**

IShipmentBookingService.cs,

ShipmentBookingService.cs

- **Domain Models:**

ShipmentBooking.cs,

Container.cs,

CargoEvent.cs,

FreightInvoice.cs

- **ViewModels:**

ShipperDashboardViewModel.cs,

BookingViewModel.cs,

BookingDetailsViewModel.cs

- **Views:**

Views/ShipmentBooking/Index.cshtml,

Create.cshtml,

Edit.cshtml,

Details.cshtml

- **Enums:**

BookingStatus.cs

- **Database Table:** ShipmentBookings

 Code Implementation & Data Flow

1. Creation & Reference Generation (ShipmentBookingService.CreateBookingAsync):

- Validates that `originPort != destinationPort` and `bookingDate >=.UtcNow.Date`.

- Generates a unique reference number via

NumberGeneratorHelper.cs (BKG- + Random String).

- Saves record with status BookingStatus.PENDING.

2. User Isolation & Security:

- GetCurrentUserId() resolves the authenticated ClaimTypes.NameIdentifier.
- All query actions enforce booking.userId == currentUserId. If unauthorized, returns NotFound() to prevent resource enumeration.

3. Document Upload Engine (ShipmentBookingController.UploadDocument):

- Validates extension against .pdf, .jpg, .png, .docx.
- Validates file size (file.Length <= 5MB).
- Sanitizes filename via Path.GetFileName(file.FileName), generates a GUID filename ({Guid} {extension}), and writes file to disk at wwwroot/uploads/bookings/{bookingId}/.
- Appends metadata JSON payload to docs_metadata.json.

4. Container Allocation Module (Container)

Enables Freight Forwarders to allocate physical shipping containers to pending bookings and manage container inventory.

 Code File Inventory

- **Controller:**

ContainerController.cs

- **Service & Interface:**

IContainerService.cs,

ContainerService.cs

- **Domain Models:**

Container.cs,

ShipmentBooking.cs

- **ViewModels:**

ForwarderDashboardViewModel.cs,

ContainerAllocationViewModel.cs

- **Views:**

Views/Container/Index.cshtml,

Allocate.cshtml,

Details.cshtml

- **Enums:**

ContainerType.cs,

ContainerStatus.cs

- **Database Table:** Containers



Code Implementation & Data Flow

1. **Container Allocation (ContainerService.AllocateContainerAsync):**

- Validates ISO container number uniqueness (`_context.Containers.AnyAsync(c => c.containerNumber == model.containerNumber)`).
- Binds containerType (Standard20ft, Standard40ft, Reefer, FlatRack, OpenTop) and sets containerStatus (e.g. LOADED).
- Automatically queries the parent ShipmentBooking and advances its status from PENDING to BookingStatus.CONFIRMED.

5. Customs Clearance Module (Customs)

Manages customs declarations, HS tariff classifications, automatic duty computation, and broker inspection workflows.



Code File Inventory

- **Controller:**

CustomsController.cs

- **Service & Interface:**

ICustomsService.cs,

CustomsService.cs

- **Domain Models:**

CustomsDeclaration.cs,

ShipmentBooking.cs

- **ViewModels:**

CustomsDashboardViewModel.cs,

FileDeclarationViewModel.cs

- **Views:**

Views/Customs/Index.cshtml,

FileDeclaration.cshtml,

Review.cshtml

- **Enums:**

DeclarationType.cs,

ClearanceStatus.cs

- **Database Table:** CustomsDeclarations

Code Implementation & Data Flow

1. **Duty Calculation Engine (CustomsService.CalculateDuty):**

- Calculates duty dynamically based on declaredValue:
 - Standard Tariff: \$5.0%\$ (declaredValue * 0.05m)
 - Preferential Tariff (specific HS codes): \$2.5%\$ (declaredValue * 0.025m)

2. **Clearance Review Workflow (CustomsService.UpdateClearanceStatusAsync):**

- Broker transitions clearance status: FILED → UNDER_REVIEW → (CLEARED, HELD, or REJECTED).
- Enforces a 1-to-1 relationship between ShipmentBooking and CustomsDeclaration.

6. Port Operations & Event Logging Module (PortOperator)

Handles real-time terminal operations, container milestone logging, and container location updates.

Code File Inventory

- **Controller:**

PortOperatorController.cs

- **Service & Interface:**

IPortOperatorService.cs,

PortOperatorService.cs

- **Domain Models:**

CargoEvent.cs,

Container.cs

- **ViewModels:**

PortDashboardViewModel.cs,

RecordMilestoneViewModel.cs

- **Views:**

Views/PortOperator/Index.cshtml,

RecordMilestone.cshtml,

ContainerDetails.cshtml

- **Enums:**

CargoEventType.cs

- **Database Table:** CargoEvents

Code Implementation & Data Flow

1. Milestone Registration (PortOperatorService.RecordCargoEventAsync):

- Records CargoEvent (GATE_IN, LOADED, SAILED, DISCHARGED, GATE_OUT) with eventTimestamp, location, description, and recordedBy.
- Synchronously updates Container.currentLocation and ContainerStatus:
 - GATE_IN → LOADED
 - SAILED → IN_TRANSIT
 - DISCHARGED → DISCHARGED (demurrage tracking starts)

7. Consignee & Live Tracking Module (Consignee, Tracking)

Provides public search and consignee-specific visibility into container locations and tracking event timelines.

Code File Inventory

- **Controllers:**

TrackingController.cs,

ConsigneeController.cs

- **Service & Interface:**

ITrackingService.cs,

TrackingService.cs

- **ViewModels:**

TrackingDetailsViewModel.cs,

TrackingTimelineItem.cs

- **Views:**

Views/Tracking/Search.cshtml,

Details.cshtml,

Views/Consignee/Index.cshtml,

MyShipments.cshtml

Code Implementation & Data Flow

1. Search Resolution (TrackingService.GetTrackingDetailsAsync):

- Searches _context.ShipmentBookings by bookingNumber OR _context.Containers by containerNumber.
 - Joins containers, cargo events, customs declaration status, and invoice status.
 - Maps CargoEvent records into an ordered list of TrackingTimelineItem ViewModels rendered as a visual step timeline in Views/Tracking/Details.cshtml.
-

8. Freight Invoicing & Finance Module (FreightInvoice)

Calculates base freight charges, surcharges, demurrage fees, generates invoices, and handles payment settlements.

 Code File Inventory

- **Controller:**

FreightInvoiceController.cs

- **Service & Interface:**

IFreightInvoiceService.cs,

FreightInvoiceService.cs

- **Domain Model:**

FreightInvoice.cs

- **ViewModels:**

InvoiceDashboardViewModel.cs,

PaymentViewModel.cs

- **Views:**

Views/FreightInvoice/Index.cshtml,

Generate.cshtml,

Pay.cshtml,

Details.cshtml

- **Enums:**

InvoiceStatus.cs

- **Database Table:** FreightInvoices

 Code Implementation & Data Flow

1. **Demurrage Calculation Algorithm (FreightInvoiceService.CalculateDemurrageAsync):**

- Queries DISCHARGED event timestamp for containers under the booking.
- Demurrage formula:
$$\text{Days at Port} = (\text{UtcNow} - \text{Discharge Date}).\text{Days}$$
$$\text{Overdue Days} = \max(0, \text{Days at Port} - 5)$$
$$\text{Demurrage Amount} = \text{Overdue Days} \times \$150.00$$

2. **Invoice Net Total:**
$$\text{Total Amount} = \text{freightCharges} + \text{surchargeAmount} + \text{demurrageAmount}$$

3. **Payment Processing (FreightInvoiceService.ProcessPaymentAsync):**

- Binds paidByUserId, records paymentDate = DateTime.UtcNow, and updates status from ISSUED to InvoiceStatus.PAID.
-

9. Reporting & Analytics Module (Reports)

Generates multi-criteria operational and financial reports, exporting data to Excel (ClosedXML) and PDF (PDFsharp).

Code File Inventory

- **Controller:**

ReportsController.cs

- **Service & Interface:**

IReportsService.cs,

ReportsService.cs

- **ViewModels:**

ReportsViewModels.cs

- **Views:**

Views/Reports/Index.cshtml,

ViewReport.cshtml

Code Implementation & Data Flow

1. **Role Security Enforcement:**

- Action method inspects `User.IsInRole("FreightForwarder")`.
- Denies access to revenue/analytics reports (`type == "revenue" || type == "analytics"`) by returning `Forbid()`.

2. **Excel Export (ReportsController.ExportExcel):**

- Instantiates a `ClosedXML.Excel.XLWorkbook()`.
- Creates worksheets, populates header rows, applies styling, auto-fits columns, and streams out `application/vnd.openxmlformats-officedocument.spreadsheetml.sheet`.

3. **PDF Export (ReportsController.ExportPdf):**

- Instantiates `PdfSharp.Pdf.PdfDocument()` and `PdfSharp.Drawing.XGraphics`.
- Draws header titles, summary boxes, and tabular rows using `XFont` and `XBrushes`, streaming out `application/pdf`.